PomPom: open signatures and parameterized closure

Typing, computation, elaboration, and examples 9 September 2026

Named binders replace de Bruijn indices; substitutions are capture avoiding. Rule labels are the exact Rocq constructor names. Formation premises are displayed. $k, j \in \mathbb{N}$; $\overline{n} := \mathsf{s}^n 0$; $\mathsf{id} := \lambda x.\, x$. A rule with no premises has an empty numerator.

1. Contexts, universes, dependent functions and pairs

Base calculus: [1, Fig. 1].

$$\varnothing \ \mathsf{wf}\ \texttt{wf\_nil} \qquad \frac{\Gamma\ \mathsf{wf} \qquad \Gamma \vdash A : \mathsf{Set}_k \qquad x \notin \mathsf{dom}(\Gamma)}{\Gamma, x : A\ \mathsf{wf}}\ \texttt{wf\_cons} \qquad \frac{\Gamma\ \mathsf{wf} \qquad x : A \in \Gamma}{\Gamma \vdash x : A}\ \texttt{ty\_var}$$

$$\frac{\Gamma\ \mathsf{wf}}{\Gamma \vdash \mathsf{Set}_k : \mathsf{Set}_{k+1}}\ \texttt{ty\_sort} \qquad \frac{\Gamma \vdash t : \mathsf{Set}_j \qquad j \le k}{\Gamma \vdash t : \mathsf{Set}_k}\ \texttt{ty\_cumul}$$

$$\frac{\Gamma \vdash A : \mathsf{Set}_j \qquad \Gamma, x : A \vdash B : \mathsf{Set}_k}{\Gamma \vdash \Pi x : A.\, B : \mathsf{Set}_{\max(j,k)}}\ \texttt{ty\_pi} \qquad \frac{\Gamma \vdash A : \mathsf{Set}_j \qquad \Gamma, x : A \vdash B : \mathsf{Set}_k}{\Gamma \vdash \Sigma x : A.\, B : \mathsf{Set}_{\max(j,k)}}\ \texttt{ty\_sigma}$$

$$\frac{\Gamma \vdash \Pi x : A.\, B : \mathsf{Set}_k \qquad \Gamma, x : A \vdash b : B}{\Gamma \vdash \lambda x.\, b : \Pi x : A.\, B}\ \texttt{ty\_lam}$$

$$\frac{\Gamma \vdash \Pi x : A.\, B : \mathsf{Set}_k \qquad \Gamma \vdash f : \Pi x : A.\, B \qquad \Gamma \vdash a : A}{\Gamma \vdash f\ a : B[a/x]}\ \texttt{ty\_app}$$

$$\frac{\Gamma \vdash \Sigma x : A.\, B : \mathsf{Set}_k \qquad \Gamma \vdash a : A \qquad \Gamma \vdash b : B[a/x]}{\Gamma \vdash (a, b) : \Sigma x : A.\, B}\ \texttt{ty\_pair}$$

$$\frac{\Gamma \vdash \Sigma x : A.\, B : \mathsf{Set}_k \qquad \Gamma \vdash p : \Sigma x : A.\, B}{\Gamma \vdash \mathsf{fst}\, p : A}\ \texttt{ty\_fst} \qquad \frac{\Gamma \vdash \Sigma x : A.\, B : \mathsf{Set}_k \qquad \Gamma \vdash p : \Sigma x : A.\, B}{\Gamma \vdash \mathsf{snd}\, p : B[\mathsf{fst}\, p/x]}\ \texttt{ty\_snd}$$

$$\frac{\Gamma \vdash t : A \qquad \Gamma \vdash B : \mathsf{Set}_k \qquad A \equiv B}{\Gamma \vdash t : B}\ \texttt{ty\_conv} \qquad \frac{\Gamma\ \mathsf{wf}}{\Gamma \vdash \mathbf{1} : \mathsf{Set}_k}\ \texttt{ty\_unitT} \qquad \frac{\Gamma\ \mathsf{wf}}{\Gamma \vdash \star : \mathbf{1}}\ \texttt{ty\_unit}$$

Computation and conversion. Write $\longmapsto$ for a root computation, $\rightsquigarrow$ for its compatible closure together with η contraction, and $\to$ for the operational relation in Rocq.

$$(\lambda x.\, b)\, a \longmapsto b[a/x], \qquad \mathsf{fst}(a, b) \longmapsto a, \quad \mathsf{snd}(a, b) \longmapsto b,$$
$$\lambda x.\, f\ x \rightsquigarrow f \quad (x \notin \mathsf{FV}(f)), \quad (\mathsf{fst}\, p, \mathsf{snd}\, p) \rightsquigarrow p.$$

$$\frac{t \longmapsto u}{t \rightsquigarrow u} \qquad \frac{t \rightsquigarrow u}{\mathcal{C}[t] \rightsquigarrow \mathcal{C}[u]} \qquad t \equiv t \qquad \frac{t \rightsquigarrow u}{t \equiv u} \qquad \frac{t \equiv u}{u \equiv t} \qquad \frac{t \equiv u \qquad u \equiv v}{t \equiv v}$$

Here $\mathcal{C}[-]$ ranges over single-hole term contexts, including binders. There is no equation unfolding a close type. The remaining root computations appear beside their operators below.

2. Identities, enumerations, branch tuples, and bottom

Enumeration universe: [1, Fig. 2].

$$\frac{\Gamma\ \mathsf{wf}}{\Gamma \vdash \mathsf{UId} : \mathsf{Set}_0}\ \texttt{ty\_uid} \qquad \frac{\Gamma\ \mathsf{wf}}{\Gamma \vdash \mathsf{tag}(s) : \mathsf{UId}}\ \texttt{ty\_tag} \qquad \frac{\Gamma\ \mathsf{wf}}{\Gamma \vdash \mathsf{EnumU} : \mathsf{Set}_0}\ \texttt{ty\_enumu}$$

$$\frac{\Gamma\ \mathsf{wf}}{\Gamma \vdash [] : \mathsf{EnumU}}\ \texttt{ty\_nile} \qquad \frac{\Gamma \vdash \ell : \mathsf{UId} \qquad \Gamma \vdash E : \mathsf{EnumU}}{\Gamma \vdash \ell :: E : \mathsf{EnumU}}\ \texttt{ty\_conse} \qquad \frac{\Gamma \vdash E : \mathsf{EnumU}}{\Gamma \vdash \mathsf{EnumT}\, E : \mathsf{Set}_0}\ \texttt{ty\_enumt}$$

$$\frac{\Gamma \vdash \ell : \mathsf{UId} \qquad \Gamma \vdash E : \mathsf{EnumU}}{\Gamma \vdash 0 : \mathsf{EnumT}(\ell :: E)}\ \texttt{ty\_zero} \qquad \frac{\Gamma \vdash \ell : \mathsf{UId} \qquad \Gamma \vdash E : \mathsf{EnumU} \qquad \Gamma \vdash n : \mathsf{EnumT}\, E}{\Gamma \vdash \mathsf{s}\, n : \mathsf{EnumT}(\ell :: E)}\ \texttt{ty\_succ}$$

$$\frac{\Gamma \vdash E : \mathsf{EnumU} \qquad \Gamma \vdash P : \mathsf{EnumT}\, E \to \mathsf{Set}_k}{\Gamma \vdash \mathsf{EPi}_k\, E\, P : \mathsf{Set}_k}\ \texttt{ty\_epi}$$

$$\frac{\Gamma \vdash E : \mathsf{EnumU} \qquad \Gamma \vdash P : \mathsf{EnumT}\, E \to \mathsf{Set}_k \qquad \Gamma \vdash b : \mathsf{EPi}_k\, E\, P \qquad \Gamma \vdash c : \mathsf{EnumT}\, E}{\Gamma \vdash \mathsf{switch}_k\, E\, P\, b\, c : P\, c}\ \texttt{ty\_switch}$$

s is a literal string. Identity expressions ℓ and positions n are distinct core terms.

$$\begin{aligned} \mathsf{EPi}_k\, []\, P &\longmapsto \mathbf{1}, \\ \mathsf{EPi}_k\, (\ell :: E)\, P &\longmapsto P\, 0 \times \mathsf{EPi}_k\, E\, (\lambda n.\, P(\mathsf{s}\, n)), \\ \mathsf{switch}_k\, (\ell :: E)\, P\, (b, b_s)\, 0 &\longmapsto b, \\ \mathsf{switch}_k\, (\ell :: E)\, P\, (b, b_s)\, (\mathsf{s}\, n) &\longmapsto \mathsf{switch}_k\, E\, (\lambda n.\, P(\mathsf{s}\, n))\, b_s\, n. \end{aligned}$$

Derived empty elimination.

$$\bot := \mathsf{EnumT}\, [], \qquad \mathsf{abort}_k\, A\, z := \mathsf{switch}_k\, []\, (\lambda \_.\, A)\, \star\, z.$$

$$\frac{\Gamma \vdash A : \mathsf{Set}_k \qquad \Gamma \vdash z : \bot}{\Gamma \vdash \mathsf{abort}_k\, A\, z : A}\ \texttt{abort\_typing} \qquad \bot : \mathsf{Set}_0, \quad (\Pi A : \mathsf{Set}_k.\, A) : \mathsf{Set}_{k+1}.$$

The displayed derived typing statement is unproved in Rocq.

Identity at a local position. $\mathsf{At}(E, s, n)$ records the literal identity s at position n.

$$\mathsf{At}(\mathsf{tag}(s) :: E, s, 0)\ \texttt{at\_here} \qquad \frac{\mathsf{At}(E, s, n)}{\mathsf{At}(\ell :: E, s, \mathsf{s}\, n)}\ \texttt{at\_there}$$

$$\frac{E \equiv E' \qquad n \equiv n' \qquad \mathsf{At}(E, s, n)}{\mathsf{At}(E', s, n')}\ \texttt{at\_convert}$$

3. Positive descriptions and interpretation

Indexed descriptions: [1, Fig. 4]; $'\bot$ is the added code.

$$\frac{\Gamma \vdash I : \mathsf{Set}_0}{\Gamma \vdash \mathsf{IDesc}\, I : \mathsf{Set}_1}\ \texttt{ty\_idesc} \qquad \frac{\Gamma \vdash I : \mathsf{Set}_0 \qquad \Gamma \vdash i : I}{\Gamma \vdash {'\mathsf{var}}\, i : \mathsf{IDesc}\, I}\ \texttt{ty\_ivar}$$

$$\frac{\Gamma \vdash I : \mathsf{Set}_0}{\Gamma \vdash {'\mathbf{1}} : \mathsf{IDesc}\, I}\ \texttt{ty\_i1} \qquad \frac{\Gamma \vdash I : \mathsf{Set}_0}{\Gamma \vdash {'\bot} : \mathsf{IDesc}\, I}\ \texttt{ty\_ibot}$$

$$\frac{\Gamma \vdash I : \mathsf{Set}_0 \qquad \Gamma \vdash D_1 : \mathsf{IDesc}\, I \qquad \Gamma \vdash D_2 : \mathsf{IDesc}\, I}{\Gamma \vdash D_1\, {'\times}\, D_2 : \mathsf{IDesc}\, I}\ \texttt{ty\_iprod}$$

$$\frac{\Gamma \vdash I : \mathsf{Set}_0 \qquad \Gamma \vdash S : \mathsf{Set}_0 \qquad \Gamma \vdash T : S \to \mathsf{IDesc}\, I}{\Gamma \vdash {'\Pi}\, S\, T : \mathsf{IDesc}\, I}\ \texttt{ty\_ipi}$$

$$\frac{\Gamma \vdash I : \mathsf{Set}_0 \qquad \Gamma \vdash S : \mathsf{Set}_0 \qquad \Gamma \vdash T : S \to \mathsf{IDesc}\, I}{\Gamma \vdash {'\Sigma}\, S\, T : \mathsf{IDesc}\, I}\ \texttt{ty\_isig}$$

$$\frac{\Gamma \vdash I : \mathsf{Set}_0 \qquad \Gamma \vdash E : \mathsf{EnumU} \qquad \Gamma \vdash p : \mathsf{EnumT}\, E \to \mathsf{IDesc}\, I}{\Gamma \vdash {'\sigma}\, E\, p : \mathsf{IDesc}\, I}\ \texttt{ty\_ichoice}$$

$$\frac{\Gamma \vdash I : \mathsf{Set}_0 \qquad \Gamma \vdash D : \mathsf{IDesc}\, I \qquad \Gamma \vdash X : I \to \mathsf{Set}_0}{\Gamma \vdash [\![D]\!]\, X : \mathsf{Set}_0}\ \texttt{ty\_interp}$$

$$\begin{aligned}
[\![{'\mathsf{var}}\, i]\!]\, X &\longmapsto X\, i, & [\![{'\mathbf{1}}]\!]\, X &\longmapsto \mathbf{1}, \quad [\![{'\bot}]\!]\, X \longmapsto \bot,\\
[\![D_1\, {'\times}\, D_2]\!]\, X &\longmapsto [\![D_1]\!]\, X \times [\![D_2]\!]\, X,\\
[\![{'\Pi}\, S\, T]\!]\, X &\longmapsto \Pi s : S.\, [\![T\, s]\!]\, X,\\
[\![{'\Sigma}\, S\, T]\!]\, X &\longmapsto \Sigma s : S.\, [\![T\, s]\!]\, X,\\
[\![{'\sigma}\, E\, p]\!]\, X &\longmapsto \Sigma c : \mathsf{EnumT}\, E.\, [\![p\, c]\!]\, X.
\end{aligned}$$

$$\mathsf{Def}\, I := \Pi i : I.\, \mathsf{IDesc}\, I, \qquad \mathsf{Family}\, I := I \to \mathsf{Set}_0.$$

Instantiations.

$$\begin{aligned}
[\![{'\Sigma}\, A\, (\lambda_.\, {'\mathsf{var}}\, \star)]\!]\, X &\rightsquigarrow^* A \times X\, \star,\\
[\![{'\mathsf{var}}\, \star\, {'\times}\, {'\mathsf{var}}\, \star]\!]\, X &\rightsquigarrow^* X\, \star \times X\, \star,\\
[\![{'\sigma}\, []\, p]\!]\, X &\longmapsto \Sigma c : \bot.\, [\![p\, c]\!]\, X.
\end{aligned}$$

An empty choice has an empty tag; its interpretation is not definitionally $\bot$ under these equations.

4. Structural hypotheses and the reference fixed point

Generic induction: [2]; indexed fixed point: [1, Fig. 4].

$$\mathsf{Mot}_I(X) := (\Sigma i : I.\, X\, i) \to \mathsf{Set}_0, \qquad \mathsf{Rec}_I(X, P) := \Pi i : I.\, \Pi y : X\, i.\, P(i, y).$$

$$\frac{\Gamma \vdash I : \mathsf{Set}_0 \qquad \Gamma \vdash D : \mathsf{IDesc}\, I \qquad \Gamma \vdash X : I \to \mathsf{Set}_0 \qquad \Gamma \vdash x_s : [\![D]\!]\, X \qquad \Gamma \vdash P : \mathsf{Mot}_I(X)}{\Gamma \vdash \mathsf{iAll}\, D\, X\, x_s\, P : \mathsf{Set}_0}\ \texttt{ty\_iall}$$

$$\frac{\begin{array}{c}\Gamma \vdash I : \mathsf{Set}_0 \\ \Gamma \vdash D : \mathsf{IDesc}\, I \qquad \Gamma \vdash X : I \to \mathsf{Set}_0 \qquad \Gamma \vdash P : \mathsf{Mot}_I(X) \qquad \Gamma \vdash h : \mathsf{Rec}_I(X, P) \qquad \Gamma \vdash x_s : [\![D]\!]\, X\end{array}}{\Gamma \vdash \mathsf{hyps}\, D\, X\, P\, h\, x_s : \mathsf{iAll}\, D\, X\, x_s\, P}\ \texttt{ty\_hyps}$$

For the table only, $\mathcal{A}_D(x) := \mathsf{iAll}\, D\, X\, x\, P$ and $\mathcal{H}_D(x) := \mathsf{hyps}\, D\, X\, P\, h\, x$.

D	x	$\mathcal{A}_D(x) \longmapsto$	$\mathcal{H}_D(x) \longmapsto$
$'\mathsf{var}\, j$	y	$P(j, y)$	$h\, j\, y$
$'\mathbf{1}$	$\star$	$\mathbf{1}$	$\star$
$'\bot$	z	$\mathbf{1}$	$\star$
$D_1 \,'\!\times\, D_2$	(a, b)	$\mathcal{A}_{D_1}(a) \times \mathcal{A}_{D_2}(b)$	$(\mathcal{H}_{D_1}(a), \mathcal{H}_{D_2}(b))$
$'\Pi\, S\, T$	f	$\Pi s : S.\, \mathcal{A}_{T\, s}(f\, s)$	$\lambda s.\, \mathcal{H}_{T\, s}(f\, s)$
$'\Sigma\, S\, T$	(s, x_s)	$\mathcal{A}_{T\, s}(x_s)$	$\mathcal{H}_{T\, s}(x_s)$
$'\sigma\, E\, p$	(c, x_s)	$\mathcal{A}_{p\, c}(x_s)$	$\mathcal{H}_{p\, c}(x_s)$

Reference μ^I, retained as a separate former.

$$\frac{\Gamma \vdash I : \mathsf{Set}_0 \qquad \Gamma \vdash D : \mathsf{Def}\, I}{\Gamma \vdash \mu^I D : I \to \mathsf{Set}_0}\ \texttt{ty\_mui}$$

$$\frac{\Gamma \vdash I : \mathsf{Set}_0 \qquad \Gamma \vdash D : \mathsf{Def}\, I \qquad \Gamma \vdash i : I \qquad \Gamma \vdash x_s : [\![D\, i]\!]\, (\mu^I D)}{\Gamma \vdash \mathsf{in}\, x_s : \mu^I D\, i}\ \texttt{ty\_in\_mui}$$

$$\begin{aligned}\mathsf{MuStep}_I(D, P) := \Pi i : I.\, \Pi x_s : [\![D\, i]\!]\, (\mu^I D).\\ \mathsf{iAll}\, (D\, i)\, (\mu^I D)\, x_s\, P \to P(i, \mathsf{in}\, x_s).\end{aligned}$$

$$\frac{\begin{array}{c}\Gamma \vdash I : \mathsf{Set}_0 \\ \Gamma \vdash D : \mathsf{Def}\, I \qquad \Gamma \vdash P : \mathsf{Mot}_I(\mu^I D) \qquad \Gamma \vdash s : \mathsf{MuStep}_I(D, P) \qquad \Gamma \vdash i : I \qquad \Gamma \vdash x : \mu^I D\, i\end{array}}{\Gamma \vdash \mathsf{ind}\, D\, P\, s\, i\, x : P(i, x)}\ \texttt{ty\_ind}$$

$$\begin{aligned}\mathsf{ind}\, D\, P\, s\, i\, (\mathsf{in}\, x_s) &\longmapsto s\, i\, x_s\, \big(\mathsf{hyps}\, (D\, i)\, (\mu^I D)\, P\, h\, x_s\big),\\ h &:= \lambda j\, y.\, \mathsf{ind}\, D\, P\, s\, j\, y.\end{aligned}$$

5. Parameterized closure: formation, cases, and induction

$$C_{F,G}(i) := \mathsf{close}\, F\, G\, i, \qquad X_G := \mathsf{close}\, G\, G, \qquad L_{F,G}(i) := [\![F\, i]\!]\, X_G.$$

$$\frac{\Gamma \vdash I : \mathsf{Set}_0 \qquad \Gamma \vdash F : \mathsf{Def}\, I \qquad \Gamma \vdash G : \mathsf{Def}\, I}{\Gamma \vdash \mathsf{close}\, F\, G : I \to \mathsf{Set}_0}\ \texttt{ty\_close}$$

$$\frac{\Gamma \vdash I : \mathsf{Set}_0 \qquad \Gamma \vdash F : \mathsf{Def}\, I \qquad \Gamma \vdash G : \mathsf{Def}\, I \qquad \Gamma \vdash i : I \qquad \Gamma \vdash x_s : [\![F\, i]\!]\, (\mathsf{close}\, G\, G)}{\Gamma \vdash \mathsf{in}\, x_s : \mathsf{close}\, F\, G\, i}\ \texttt{ty\_in\_close}$$

$$\frac{\begin{array}{c}\Gamma \vdash I : \mathsf{Set}_0 \qquad \Gamma \vdash F : \mathsf{Def}\, I \qquad \Gamma \vdash G : \mathsf{Def}\, I \\ \Gamma \vdash i : I \qquad \Gamma \vdash Q : C_{F,G}(i) \to \mathsf{Set}_k \qquad \Gamma \vdash b : \Pi x_s : [\![F\, i]\!]\, X_G.\, Q(\mathsf{in}\, x_s) \qquad \Gamma \vdash x : C_{F,G}(i)\end{array}}{\Gamma \vdash \mathsf{closeCase}_k\, F\, G\, i\, Q\, b\, x : Q\, x}\ \texttt{ty\_close\_case}$$

$$\mathsf{closeCase}_k\, F\, G\, i\, Q\, b\, (\mathsf{in}\, x_s) \longmapsto b\, x_s,$$

$$\mathsf{unroll}_{F,G,i}\, x := \mathsf{closeCase}_0\, F\, G\, i\, (\lambda\_.\, L_{F,G}(i))\, \mathsf{id}\, x.$$

$$\frac{\Gamma \vdash I : \mathsf{Set}_0 \qquad \Gamma \vdash F : \mathsf{Def}\, I \qquad \Gamma \vdash G : \mathsf{Def}\, I \qquad \Gamma \vdash i : I \qquad \Gamma \vdash x : C_{F,G}(i)}{\Gamma \vdash \mathsf{unroll}_{F,G,i}\, x : [\![F\, i]\!]\, X_G}\ \texttt{unroll\_typing}$$

$$\mathsf{unroll}_{F,G,i}(\mathsf{in}\, x_s) \rightsquigarrow^* x_s.$$

The derived typing statement is unproved. The outer definition F and recursive definition G are arbitrary; no row inclusion premise is imposed.

Induction motive and method, expanded.

$$\mathsf{CloseMot}_I(G) := \Pi H : \mathsf{Def}\, I.\, \Pi j : I.\, C_{H,G}(j) \to \mathsf{Set}_0,$$

$$P^G := \lambda z.\, P\, G\, (\mathsf{fst}\, z)\, (\mathsf{snd}\, z),$$

$$\mathsf{CloseStep}_I(G, P) := \Pi H : \mathsf{Def}\, I.\, \Pi j : I.\, \Pi y_s : [\![H\, j]\!]\, X_G.\ \mathsf{iAll}\, (H\, j)\, X_G\, y_s\, P^G \to P\, H\, j\, (\mathsf{in}\, y_s).$$

$$\frac{\begin{array}{c}\Gamma \vdash I : \mathsf{Set}_0 \qquad \Gamma \vdash G : \mathsf{Def}\, I \qquad \Gamma \vdash P : \Pi H : \mathsf{Def}\, I.\, \Pi j : I.\, C_{H,G}(j) \to \mathsf{Set}_0 \\ \Gamma \vdash s : \mathsf{CloseStep}_I(G, P) \qquad \Gamma \vdash F : \mathsf{Def}\, I \qquad \Gamma \vdash i : I \qquad \Gamma \vdash x : C_{F,G}(i)\end{array}}{\Gamma \vdash \mathsf{closeInd}\, G\, P\, s\, F\, i\, x : P\, F\, i\, x}\ \texttt{ty\_close\_ind}$$

$$\mathsf{closeInd}\, G\, P\, s\, F\, i\, (\mathsf{in}\, x_s) \longmapsto s\, F\, i\, x_s\, \big(\mathsf{hyps}\, (F\, i)\, X_G\, P^G\, h\, x_s\big),$$

$$h := \lambda j\, y.\, \mathsf{closeInd}\, G\, P\, s\, G\, j\, y.$$

Children use outer definition G and motive $P\, G\, j\, y$. Case motives range over Set_k; recursive motives range over Set_0. The diagonal $C_{G,G}$ is not judgmentally identified with $\mu^I G$.

6. Rows, open signatures, and generated handlers

$$
\begin{aligned}
r = [(\ell_0, D_0), \ldots, (\ell_{n-1}, D_{n-1})], \qquad & \mathsf{names}(r) = [\ell_0, \ldots, \ell_{n-1}], \\
E_r = [\mathsf{tag}(\ell_0), \ldots, \mathsf{tag}(\ell_{n-1})], \qquad & \mathsf{tuple}([]) = \star, \\
\mathsf{tuple}(t :: t_s) = (t, \mathsf{tuple}(t_s)), \qquad & d_r = \mathsf{tuple}([D_0, \ldots, D_{n-1}]), \\
p_r := \lambda c.\, \mathsf{switch}_1\, E_r\, (\lambda\_.\, \mathsf{IDesc}\, I)\, d_r\, c, \qquad & \langle r \rangle_I := {}'\sigma\, E_r\, p_r.
\end{aligned}
$$

Rows are finite lists of literal identities and core description terms.

$$
\begin{aligned}
\mathsf{RowOK}_\Gamma(I, r) \quad :\Longleftrightarrow \quad & (\Gamma \vdash I : \mathsf{Set}_0) \ \wedge \ \mathsf{NoDup}(\mathsf{names}(r)) \\
& \wedge \ \forall (\ell, D) \in r.\ \Gamma \vdash D : \mathsf{IDesc}\, I.
\end{aligned}
$$

$$
\frac{\mathsf{RowOK}_\Gamma(I, r) \qquad \Gamma \vdash D : \mathsf{IDesc}\, I \qquad D \equiv \langle r \rangle_I}{\Gamma \vdash D \Downarrow_I r}\ \texttt{view\_rows}
$$

Open signature assembly.

$$
\mathsf{signature}_I(r) := \lambda i.\, \langle r_i \rangle_I, \qquad r_i \text{ is scoped in } \Gamma, i : I.
$$

$$
\frac{\Gamma \vdash I : \mathsf{Set}_0 \qquad \mathsf{RowOK}_{\Gamma, i:I}(I, r_i)}{\Gamma \vdash \lambda i.\, \langle r_i \rangle_I : \mathsf{Def}\, I}\ \texttt{signature\_typing}
$$

This derived statement is unproved. Surface $\{R :: \mathsf{branches}\, R\}$ uses positive slots: $R\, j$ becomes ${}'\mathsf{var}\, j$. The Rocq relation starts with those translated schemes.

Dispatcher and branch handlers (core terms).

$$
\begin{aligned}
M_{I,r,X,Y} &:= \lambda c.\, [\![p_r\, c]\!]\, X \to Y, \\
\mathsf{dispatch}^k_{I,r,X,Y}(h_s) &:= \lambda z.\, \big(\mathsf{switch}_k\, E_r\, M_{I,r,X,Y}\, (\mathsf{tuple}(h_s))\, (\mathsf{fst}\, z)\big)(\mathsf{snd}\, z), \\
\mathsf{retag}_m &:= \lambda x_s.\, (\overline{m}, x_s), \\
\mathsf{exfalso}^k_Y(d) &:= \lambda x_s.\, \mathsf{abort}_k\, Y\, (d\, x_s).
\end{aligned}
$$

X is the recursive family; Y is the handler result type. The switch motive depends on the source tag.

Row handler relation. $\mathsf{Handlers}_{\Gamma,I}(X, Y; r'; r) \rightsquigarrow h_s$ aligns source r with target r'.

$$
\mathsf{Handlers}_{\Gamma,I}(X, Y; r'; []) \rightsquigarrow [] \ \texttt{rh\_nil}
$$

$$
\frac{r'[m] = (\ell, D') \qquad [\![D]\!]\, X \equiv [\![D']\!]\, X \qquad \mathsf{Handlers}_{\Gamma,I}(X, Y; r'; r) \rightsquigarrow h_s}{\mathsf{Handlers}_{\Gamma,I}(X, Y; r'; (\ell, D) :: r) \rightsquigarrow \mathsf{retag}_m :: h_s}\ \texttt{rh\_live}
$$

$$
\frac{\Gamma \vdash D\ \mathsf{dead}^I_X \rightsquigarrow d \qquad \mathsf{Handlers}_{\Gamma,I}(X, Y; r'; r) \rightsquigarrow h_s}{\mathsf{Handlers}_{\Gamma,I}(X, Y; r'; (\ell, D) :: r) \rightsquigarrow \mathsf{exfalso}^0_Y(d) :: h_s}\ \texttt{rh\_dead}
$$

A live handler preserves the payload and translates the local tag. A dead handler requires no occurrence of ℓ in r'. Row and endpoint formation checks are supplied by `ds_rows` below.

7. Certified empty descriptions and description coercions

$\Gamma \vdash D\ \mathsf{dead}^I_X \rightsquigarrow d$ generates a witness $d : [\![D]\!]\, X \to \bot$.

$$\frac{\Gamma \vdash I : \mathsf{Set}_0 \qquad \Gamma \vdash D : \mathsf{IDesc}\, I \qquad \Gamma \vdash X : I \to \mathsf{Set}_0 \qquad D \equiv {}'\bot}{\Gamma \vdash D\ \mathsf{dead}^I_X \rightsquigarrow \mathsf{id}}\ \texttt{dead\_bot}$$

$$\frac{\Gamma \vdash I : \mathsf{Set}_0 \qquad \Gamma \vdash D : \mathsf{IDesc}\, I \qquad \Gamma \vdash X : I \to \mathsf{Set}_0 \qquad D \equiv {}'\sigma\, []\, p}{\Gamma \vdash D\ \mathsf{dead}^I_X \rightsquigarrow \lambda z.\, \mathsf{fst}\, z}\ \texttt{dead\_nil}$$

$$\frac{\Gamma \vdash I : \mathsf{Set}_0 \qquad \Gamma \vdash D : \mathsf{IDesc}\, I \qquad \Gamma \vdash X : I \to \mathsf{Set}_0 \qquad D \equiv D_1 \,{}'\!\times D_2 \qquad \Gamma \vdash D_1\ \mathsf{dead}^I_X \rightsquigarrow d}{\Gamma \vdash D\ \mathsf{dead}^I_X \rightsquigarrow \lambda z.\, d(\mathsf{fst}\, z)}\ \texttt{dead\_left}$$

$$\frac{\Gamma \vdash I : \mathsf{Set}_0 \qquad \Gamma \vdash D : \mathsf{IDesc}\, I \qquad \Gamma \vdash X : I \to \mathsf{Set}_0 \qquad D \equiv D_1 \,{}'\!\times D_2 \qquad \Gamma \vdash D_2\ \mathsf{dead}^I_X \rightsquigarrow d}{\Gamma \vdash D\ \mathsf{dead}^I_X \rightsquigarrow \lambda z.\, d(\mathsf{snd}\, z)}\ \texttt{dead\_right}$$

$$\frac{\Gamma \vdash I : \mathsf{Set}_0 \qquad \Gamma \vdash D : \mathsf{IDesc}\, I \qquad \Gamma \vdash X : I \to \mathsf{Set}_0 \qquad \Gamma \vdash D \Downarrow_I r \qquad \mathsf{DeadRows}_{\Gamma,I}(r, X) \rightsquigarrow h_s}{\Gamma \vdash D\ \mathsf{dead}^I_X \rightsquigarrow \mathsf{dispatch}^0_{I,r,X,\bot}(h_s)}\ \texttt{dead\_choice}$$

$$\frac{\Gamma \vdash I : \mathsf{Set}_0 \qquad \Gamma \vdash D : \mathsf{IDesc}\, I \qquad \Gamma \vdash X : I \to \mathsf{Set}_0 \qquad \Gamma \vdash d : [\![D]\!]\, X \to \bot}{\Gamma \vdash D\ \mathsf{dead}^I_X \rightsquigarrow d}\ \texttt{dead\_provided}$$

$$\mathsf{DeadRows}_{\Gamma,I}([], X) \rightsquigarrow []\ \texttt{dr\_nil} \qquad \frac{\Gamma \vdash D\ \mathsf{dead}^I_X \rightsquigarrow d \qquad \mathsf{DeadRows}_{\Gamma,I}(r, X) \rightsquigarrow d_s}{\mathsf{DeadRows}_{\Gamma,I}((\ell, D) :: r, X) \rightsquigarrow d :: d_s}\ \texttt{dr\_cons}$$

An explicit empty-payload witness may be checked; non-reduction is not an emptiness test.

Description coercions. $\Gamma \vdash D \preceq^I_X D' \rightsquigarrow q$ generates $q : [\![D]\!]\, X \to [\![D']\!]\, X$.

$$\frac{\Gamma \vdash I : \mathsf{Set}_0 \qquad \Gamma \vdash D : \mathsf{IDesc}\, I \qquad \Gamma \vdash D' : \mathsf{IDesc}\, I \qquad \Gamma \vdash X : I \to \mathsf{Set}_0 \qquad D \equiv D'}{\Gamma \vdash D \preceq^I_X D' \rightsquigarrow \mathsf{id}}\ \texttt{ds\_conv}$$

$$\frac{\Gamma \vdash D\ \mathsf{dead}^I_X \rightsquigarrow d \qquad \Gamma \vdash I : \mathsf{Set}_0 \qquad \Gamma \vdash D' : \mathsf{IDesc}\, I \qquad \Gamma \vdash X : I \to \mathsf{Set}_0}{\Gamma \vdash D \preceq^I_X D' \rightsquigarrow \mathsf{exfalso}^0_{[\![D']\!]\, X}(d)}\ \texttt{ds\_dead}$$

$$\frac{\begin{array}{c}\Gamma \vdash I : \mathsf{Set}_0 \qquad \Gamma \vdash D : \mathsf{IDesc}\, I \qquad \Gamma \vdash D' : \mathsf{IDesc}\, I \\ \Gamma \vdash X : I \to \mathsf{Set}_0 \qquad \Gamma \vdash D \Downarrow_I r \qquad \Gamma \vdash D' \Downarrow_I r' \qquad \mathsf{Handlers}_{\Gamma,I}(X, [\![D']\!]\, X; r'; r) \rightsquigarrow h_s\end{array}}{\Gamma \vdash D \preceq^I_X D' \rightsquigarrow \mathsf{dispatch}^0_{I,r,X,[\![D']\!]\, X}(h_s)}\ \texttt{ds\_rows}$$

8. Type coercions

Outer-constructor restriction is motivated by [3, Section 2]; these coercion rules are the revision. $\Gamma \vdash A \sqsubseteq B \rightsquigarrow c$ generates a core function $c : A \to B$.

$$\frac{\Gamma \vdash A : \mathsf{Set}_j \qquad \Gamma \vdash B : \mathsf{Set}_k \qquad A \equiv B}{\Gamma \vdash A \sqsubseteq B \rightsquigarrow \mathsf{id}}\ \texttt{su\_conv} \qquad \frac{\Gamma \vdash A \sqsubseteq B \rightsquigarrow c \qquad \Gamma \vdash B \sqsubseteq C \rightsquigarrow d}{\Gamma \vdash A \sqsubseteq C \rightsquigarrow \lambda x.\, d(c\,x)}\ \texttt{su\_trans}$$

$$\frac{\Gamma \vdash A : \mathsf{Set}_k}{\Gamma \vdash \bot \sqsubseteq A \rightsquigarrow \lambda z.\, \mathsf{abort}_k\, A\, z}\ \texttt{su\_bottom}$$

$$\frac{\Gamma \vdash \Pi a : A.\, B : \mathsf{Set}_j \qquad \Gamma \vdash \Pi x : A'.\, B' : \mathsf{Set}_k \qquad \Gamma \vdash A' \sqsubseteq A \rightsquigarrow c \qquad \Gamma, x : A' \vdash B[c\,x/a] \sqsubseteq B' \rightsquigarrow d}{\Gamma \vdash (\Pi a : A.\, B) \sqsubseteq (\Pi x : A'.\, B') \rightsquigarrow \lambda f\, x.\, d(f(c\,x))}\ \texttt{su\_pi}$$

$$\frac{\begin{array}{c}\Gamma \vdash I : \mathsf{Set}_0 \\ \Gamma \vdash F : \mathsf{Def}\, I \qquad \Gamma \vdash H : \mathsf{Def}\, I \qquad \Gamma \vdash G : \mathsf{Def}\, I \qquad \Gamma \vdash i : I \qquad \Gamma \vdash F\, i \preceq^{I}_{\mathsf{close}\, G\, G}\, H\, i \rightsquigarrow q\end{array}}{\Gamma \vdash C_{F,G}(i) \sqsubseteq C_{H,G}(i) \rightsquigarrow \lambda x.\, \mathsf{in}\big(q(\mathsf{unroll}_{F,G,i}\, x)\big)}\ \texttt{su\_close}$$

Repeated identical formation premises have been combined. The recursive definition is the same G on both sides of `su_close`.

Dependent-function instantiation. For element type S, instantiate `su_pi` as follows:

$$\begin{gathered}\Gamma \vdash S : \mathsf{Set}_0, \qquad \Gamma \vdash \Pi a : \mathsf{List}\, S.\, B(a) : \mathsf{Set}_j, \qquad \Gamma \vdash \Pi x : \mathsf{NonEmpty}\, S.\, B'(x) : \mathsf{Set}_k, \\ \Gamma \vdash \mathsf{NonEmpty}\, S \sqsubseteq \mathsf{List}\, S \rightsquigarrow \mathsf{toList}, \\ \Gamma, x : \mathsf{NonEmpty}\, S \vdash B(\mathsf{toList}\, x) \sqsubseteq B'(x) \rightsquigarrow d_x, \\ (\lambda f\, x.\, d_x(f(\mathsf{toList}\, x))) : (\Pi a : \mathsf{List}\, S.\, B(a)) \to (\Pi x : \mathsf{NonEmpty}\, S.\, B'(x)).\end{gathered}$$

The source codomain is $B(\mathsf{toList}\, x)$, not $B(x)$.

Rocq binder instantiation. For de Bruijn terms, $\uparrow_c^m t$ raises indices at cutoff c by m; $[u/0]t$ substitutes for index 0.

$$\begin{aligned}\mathsf{coercedCodomain}(c, B) &:= [\,(\uparrow_0^1 c)\,0/0\,](\uparrow_1^1 B), \\ \mathsf{piCoercion}(c, d) &:= \lambda.\lambda.\, (\uparrow_1^1 d)\big(1\,((\uparrow_0^2 c)\,0)\big).\end{aligned}$$

The lifted d remains under the new argument binder; the function binder is inserted below it. For outer-context variable $c = 0$ and $B = 1\,0$:

$$\mathsf{coercedCodomain}(0, 1\,0) = 1\,(1\,0).$$

9. Source synthesis and checking

$\Gamma \vdash e \Rightarrow A \rightsquigarrow t$ synthesizes A and generates t; $\Gamma \vdash e \Leftarrow A \rightsquigarrow t$ checks against A and generates t.

$$\frac{\Gamma\ \mathsf{wf} \qquad x : A \in \Gamma}{\Gamma \vdash x \Rightarrow A \rightsquigarrow x}\ \texttt{es\_var} \qquad \frac{\Gamma \vdash A : \mathsf{Set}_k \qquad \Gamma \vdash e \Leftarrow A \rightsquigarrow t}{\Gamma \vdash (e : A) \Rightarrow A \rightsquigarrow t}\ \texttt{es\_ann}$$

$$\frac{\Gamma \vdash \Pi x : A.\, B : \mathsf{Set}_k \qquad \Gamma \vdash f \Rightarrow \Pi x : A.\, B \rightsquigarrow t \qquad \Gamma \vdash a \Leftarrow A \rightsquigarrow u}{\Gamma \vdash f\, a \Rightarrow B[u/x] \rightsquigarrow t\, u}\ \texttt{es\_app}$$

$$\frac{\Gamma \vdash \Pi x : A.\, B : \mathsf{Set}_k \qquad \Gamma, x : A \vdash e \Leftarrow B \rightsquigarrow t}{\Gamma \vdash \lambda x.\, e \Leftarrow \Pi x : A.\, B \rightsquigarrow \lambda x.\, t}\ \texttt{ec\_lam}$$

$$\frac{\Gamma \vdash \Sigma x : A.\, B : \mathsf{Set}_k \qquad \Gamma \vdash e \Leftarrow A \rightsquigarrow t \qquad \Gamma \vdash f \Leftarrow B[t/x] \rightsquigarrow u}{\Gamma \vdash (e, f) \Leftarrow \Sigma x : A.\, B \rightsquigarrow (t, u)}\ \texttt{ec\_pair}$$

$$\frac{\Gamma \vdash t : A}{\Gamma \vdash \mathsf{core}(t) \Leftarrow A \rightsquigarrow t}\ \texttt{ec\_core}$$

$$\frac{\Gamma \vdash e \Rightarrow A \rightsquigarrow t \qquad \Gamma \vdash B : \mathsf{Set}_k \qquad A \equiv B}{\Gamma \vdash e \Leftarrow B \rightsquigarrow t}\ \texttt{ec\_conversion}$$

$$\frac{\Gamma \vdash e \Leftarrow A \rightsquigarrow t \qquad \Gamma \vdash B : \mathsf{Set}_k \qquad A \equiv B}{\Gamma \vdash e \Leftarrow B \rightsquigarrow t}\ \texttt{ec\_target\_conversion}$$

$$\frac{\Gamma \vdash e \Rightarrow A \rightsquigarrow t \qquad \Gamma \vdash A \sqsubseteq B \rightsquigarrow c}{\Gamma \vdash e \Leftarrow B \rightsquigarrow c\, t}\ \texttt{ec\_subsumption}$$

$$\frac{\Gamma \vdash I : \mathsf{Set}_0 \qquad \mathsf{RowOK}_{\Gamma, i:I}(I, r_i)}{\Gamma \vdash \mathsf{signature}\ I\ r \Rightarrow \mathsf{Def}\ I \rightsquigarrow \lambda i.\, \langle r_i \rangle_I}\ \texttt{es\_signature}$$

$$\frac{\Gamma \vdash I : \mathsf{Set}_0 \qquad \Gamma \vdash F \Leftarrow \mathsf{Def}\ I \rightsquigarrow f \qquad \Gamma \vdash G \Leftarrow \mathsf{Def}\ I \rightsquigarrow g}{\Gamma \vdash F(G) \Rightarrow I \to \mathsf{Set}_0 \rightsquigarrow \mathsf{close}\, f\, g}\ \texttt{es\_close}$$

For unindexed surface datatypes, the unit application is inserted: $F(G)$ abbreviates $(\mathsf{close}\, f\, g)\star$. An embedded core term synthesizes only through an annotation. These are relations, not a deterministic inference algorithm.

10. Named constructors and exhaustive cases

$$\frac{\begin{array}{c}\Gamma \vdash I : \mathsf{Set}_0 \qquad \Gamma \vdash F : \mathsf{Def}\, I \\ \Gamma \vdash G : \mathsf{Def}\, I \qquad \Gamma \vdash i : I \qquad \Gamma \vdash F\, i \Downarrow_I r \qquad r[n] = (\ell, D) \qquad \Gamma \vdash e \Leftarrow [\![D]\!]\, X_G \rightsquigarrow x_s\end{array}}{\Gamma \vdash \ell(e) \Leftarrow C_{F,G}(i) \rightsquigarrow \mathsf{in}(\overline{n}, x_s)}\ \texttt{ec\_constructor}$$

The expected type determines the row, local tag, and recursive payload.

Clause compilation. Let b_s be a list of clauses $(\ell, y.e)$, with y bound in e. $\mathsf{names}(b_s)$ is their list of identities. $\mathsf{Cases}_{\Gamma,k,I}(X, Q; b_s; r) \rightsquigarrow h_s$ compiles one handler for each entry of r.

$$\mathsf{Cases}_{\Gamma,k,I}(X, Q; b_s; []) \rightsquigarrow [] \ \texttt{cases\_nil}$$

$$\frac{(\ell, y.e) \in b_s \qquad \Gamma, y : [\![D]\!]\, X \vdash e \Leftarrow Q \rightsquigarrow t \qquad \mathsf{Cases}_{\Gamma,k,I}(X, Q; b_s; r) \rightsquigarrow h_s}{\mathsf{Cases}_{\Gamma,k,I}(X, Q; b_s; (\ell, D) :: r) \rightsquigarrow (\lambda y.\, t) :: h_s}\ \texttt{cases\_live}$$

$$\frac{\ell \notin \mathsf{names}(b_s) \qquad \Gamma \vdash D\ \mathsf{dead}^I_X \rightsquigarrow d \qquad \mathsf{Cases}_{\Gamma,k,I}(X, Q; b_s; r) \rightsquigarrow h_s}{\mathsf{Cases}_{\Gamma,k,I}(X, Q; b_s; (\ell, D) :: r) \rightsquigarrow \mathsf{exfalso}^k_Q(d) :: h_s}\ \texttt{cases\_dead}$$

Q is scoped in Γ, so y does not occur free in Q. An omitted clause needs a checked dead witness.

Case synthesis.

$$\frac{\begin{array}{c}\Gamma \vdash I : \mathsf{Set}_0 \qquad \Gamma \vdash F : \mathsf{Def}\, I \qquad \Gamma \vdash G : \mathsf{Def}\, I \qquad \Gamma \vdash i : I \qquad \Gamma \vdash e \Rightarrow C_{F,G}(i) \rightsquigarrow x \qquad \Gamma \vdash Q : \mathsf{Set}_k \\ \Gamma \vdash F\, i \Downarrow_I r \qquad \mathsf{NoDup}(\mathsf{names}(b_s)) \qquad \forall (\ell, y.e_\ell) \in b_s.\ \ell \in \mathsf{names}(r) \qquad \mathsf{Cases}_{\Gamma,k,I}(X_G, Q; b_s; r) \rightsquigarrow h_s\end{array}}{\Gamma \vdash \mathsf{case}_Q\ e\ \mathsf{of}\ b_s \Rightarrow Q \rightsquigarrow \mathsf{closeCase}_k\, F\, G\, i\, (\lambda\_.\, Q)\, (\mathsf{dispatch}^k_{I,r,X_G,Q}(h_s))\, x}\ \texttt{es\_case}$$

The row view supplies distinct constructor identities. The two clause premises reject duplicate and foreign identities.

Single live clause instantiation. For $r = [(\mathsf{cons}, K_A)]$, $X = X_{L_A}$, and $b_s = [(\mathsf{cons}, y.\mathsf{core}(\mathsf{fst}\, y))]$:

$$[\![K_A]\!]\, X_{L_A} \equiv A \times \mathsf{List}\, A, \qquad \Gamma, y : A \times \mathsf{List}\, A \vdash \mathsf{fst}\, y : A,$$
$$\mathsf{Cases}_{\Gamma,k,I}(X_{L_A}, A; b_s; [(\mathsf{cons}, K_A)]) \rightsquigarrow [\lambda y.\, \mathsf{fst}\, y],$$
$$\mathsf{case}_A\ e\ \mathsf{of}\ b_s \rightsquigarrow \mathsf{closeCase}_0\, U_A\, L_A \star (\lambda\_.\, A)\, \big(\mathsf{dispatch}^0_{\mathbf{1},r,X_{L_A},A}([\lambda y.\, \mathsf{fst}\, y])\big)\, x.$$

Here e synthesizes $\mathsf{NonEmpty}\, A$ and generates x; L_A, U_A, K_A are defined next.

11. Lists, nonempty lists, and constructor reuse

Throughout this example, $\Gamma \vdash A : \mathsf{Set}_0$ and $I = \mathbf{1}$. The names $\mathsf{nil}, \mathsf{cons}, \mathsf{fork}$ are literal constructor identities.

$$N := \prime\mathbf{1}, \qquad K_A := \prime\Sigma\, A\, (\lambda\_.\, \prime\mathsf{var}\star), \qquad B := \prime\mathsf{var}\ \star\, \prime\times \prime\mathsf{var}\star,$$
$$r_L := [(\mathsf{nil}, N), (\mathsf{cons}, K_A)], \quad r_U := [(\mathsf{cons}, K_A)],$$
$$r_P := [(\mathsf{nil}, \prime\bot), (\mathsf{cons}, K_A)], \quad r_T := [(\mathsf{nil}, N), (\mathsf{cons}, K_A), (\mathsf{fork}, B)].$$

$$L_A := \lambda i : \mathbf{1}.\, \langle r_L \rangle_I, \qquad U_A := \lambda i : \mathbf{1}.\, \langle r_U \rangle_I,$$
$$P_A := \lambda i : \mathbf{1}.\, \langle r_P \rangle_I, \qquad T_A := \lambda i : \mathbf{1}.\, \langle r_T \rangle_I,$$
$$\mathsf{List}\, A := C_{L_A, L_A}(\star), \quad \mathsf{NonEmpty}\, A := C_{U_A, L_A}(\star),$$
$$\mathsf{Padded}\, A := C_{P_A, L_A}(\star), \quad \mathsf{Tree}\, A := C_{T_A, T_A}(\star).$$

Surface forms expose the recursive parameter:

$$L_A = \{R :: \mathsf{nil} : \mathbf{1},\ \mathsf{cons} : A \times R\},$$
$$U_A = \{R :: \mathsf{cons} : A \times R\},$$
$$T_A = \{R :: \mathsf{nil} : \mathbf{1},\ \mathsf{cons} : A \times R,\ \mathsf{fork} : R \times R\},$$
$$\mathsf{List}\, A = L_A(L_A), \qquad \mathsf{NonEmpty}\, A = U_A(L_A), \qquad \mathsf{Tree}\, A = T_A(T_A).$$

Constructor-rule instantiations.

Expected type	Identity	Position	Payload type
$\mathsf{List}\, A$	nil	0	$\mathbf{1}$
$\mathsf{List}\, A$	cons	$\mathsf{s}\,0$	$A \times \mathsf{List}\, A$
$\mathsf{NonEmpty}\, A$	cons	0	$A \times \mathsf{List}\, A$
$\mathsf{Tree}\, A$	cons	$\mathsf{s}\,0$	$A \times \mathsf{Tree}\, A$
$\mathsf{Tree}\, A$	fork	$\mathsf{s}(\mathsf{s}\,0)$	$\mathsf{Tree}\, A \times \mathsf{Tree}\, A$

$$\frac{\Gamma \vdash \mathbf{1} : \mathsf{Set}_0 \qquad \Gamma \vdash L_A : \mathsf{Def}\,\mathbf{1} \qquad \Gamma \vdash \star : \mathbf{1} \qquad \Gamma \vdash a : A \qquad \Gamma \vdash x_s : \mathsf{List}\, A}{\Gamma \vdash \mathsf{in}(\mathsf{s}\,0, (a, x_s)) : C_{L_A, L_A}(\star)}\ \texttt{List-cons}$$

$$\frac{\Gamma \vdash \mathbf{1} : \mathsf{Set}_0 \qquad \Gamma \vdash U_A : \mathsf{Def}\,\mathbf{1} \qquad \Gamma \vdash L_A : \mathsf{Def}\,\mathbf{1} \qquad \Gamma \vdash \star : \mathbf{1} \qquad \Gamma \vdash a : A \qquad \Gamma \vdash x_s : \mathsf{List}\, A}{\Gamma \vdash \mathsf{in}(0, (a, x_s)) : C_{U_A, L_A}(\star)}\ \texttt{NonEmpty-cons}$$

These are derived instantiations using interpretation, pair formation, and `ty_in_close`; their general typing claims remain unproved.

$$\mathsf{nil} := \mathsf{in}(0, \star) : \mathsf{List}\, A,$$
$$\mathsf{cons}_L(a, x_s) := \mathsf{in}(\mathsf{s}\,0, (a, x_s)) : \mathsf{List}\, A,$$
$$\mathsf{cons}_U(a, x_s) := \mathsf{in}(0, (a, x_s)) : \mathsf{NonEmpty}\, A,$$
$$\mathsf{singleton}\, a := \mathsf{cons}_U(a, \mathsf{nil}).$$

12. Concrete widening, observers, and induction

Widening only the outer tag.

$$
\begin{aligned}
q_{U,L} &:= \mathsf{dispatch}^0_{\mathbf{1},r_U,X_{L_A},[\![L_A\,\star]\!]\,X_{L_A}}([\mathsf{retag}_1]),\\
\mathsf{toList} &:= \lambda x.\,\mathsf{in}\big(q_{U,L}(\mathsf{unroll}_{U_A,L_A,\star}\,x)\big),\\
q_{U,L}(0,(a,x_s)) &\rightsquigarrow^* (\mathsf{s}\,0,(a,x_s)),\\
\mathsf{toList}(\mathsf{cons}_U(a,x_s)) &\rightsquigarrow^* \mathsf{cons}_L(a,x_s).
\end{aligned}
$$

$$
\frac{\Gamma \vdash U_A : \mathsf{Def}\,\mathbf{1} \qquad \Gamma \vdash L_A : \mathsf{Def}\,\mathbf{1} \qquad \Gamma \vdash \star : \mathbf{1} \qquad \Gamma \vdash U_A\star \preceq^{\mathbf{1}}_{X_{L_A}} L_A\star \rightsquigarrow q_{U,L} \qquad \Gamma \vdash \mathbf{1} : \mathsf{Set}_0}{\Gamma \vdash \mathsf{NonEmpty}\,A \sqsubseteq \mathsf{List}\,A \rightsquigarrow \mathsf{toList}}\ \texttt{NonEmpty-sub-List}
$$

Bottom-padded and compact rows. For each conversion, $X = X_{L_A}$ and Y is the target layer type.

Source row	Target row	Handler list
r_U	r_L	$[\mathsf{retag}_1]$
r_U	r_P	$[\mathsf{retag}_1]$
r_P	r_U	$[\mathsf{exfalso}^0_{[\![U_A\,\star]\!]\,X_{L_A}}(\mathsf{id}), \mathsf{retag}_0]$

Use each dispatcher inside $\lambda x.\,\mathsf{in}(q(\mathsf{unroll}\,x))$. The padded nil branch has witness $\mathsf{id} : \bot \to \bot$.

Observers via one-layer elimination. For $r = r_U$ and $X = X_{L_A}$,

$$
\begin{aligned}
\mathsf{head} &:= \lambda x.\,\mathsf{closeCase}_0\,U_A\,L_A\,\star\,(\lambda\_.\,A)\,\big(\mathsf{dispatch}^0_{\mathbf{1},r,X,A}([\lambda y.\,\mathsf{fst}\,y])\big)\,x,\\
\mathsf{tail} &:= \lambda x.\,\mathsf{closeCase}_0\,U_A\,L_A\,\star\,(\lambda\_.\,\mathsf{List}\,A)\,\big(\mathsf{dispatch}^0_{\mathbf{1},r,X,\mathsf{List}\,A}([\lambda y.\,\mathsf{snd}\,y])\big)\,x,\\
\mathsf{head}(\mathsf{singleton}\,a) &\rightsquigarrow^* a, \qquad \mathsf{tail}(\mathsf{singleton}\,a) \rightsquigarrow^* \mathsf{nil}.
\end{aligned}
$$

Instantiating the induction hypotheses. Let $P : \mathsf{CloseMot}_{\mathbf{1}}(L_A)$ and $s : \mathsf{CloseStep}_{\mathbf{1}}(L_A, P)$. Put $h := \lambda j\,y.\,\mathsf{closeInd}\,L_A\,P\,s\,L_A\,j\,y$.

$$
\begin{aligned}
\mathsf{iAll}\,(L_A\,\star)\,X_{L_A}\,(0,\star)\,P^{L_A} &\rightsquigarrow^* \mathbf{1},\\
\mathsf{iAll}\,(L_A\,\star)\,X_{L_A}\,(\mathsf{s}\,0,(a,x_s))\,P^{L_A} &\rightsquigarrow^* P\,L_A\,\star\,x_s,\\
\mathsf{iAll}\,(U_A\,\star)\,X_{L_A}\,(0,(a,x_s))\,P^{L_A} &\rightsquigarrow^* P\,L_A\,\star\,x_s,\\
\mathsf{closeInd}\,L_A\,P\,s\,U_A\,\star\,(\mathsf{cons}_U(a,x_s)) &\rightsquigarrow^* s\,U_A\,\star\,(0,(a,x_s))\,(h\,\star\,x_s).
\end{aligned}
$$

The nonempty result uses $P\,U_A$, while the tail hypothesis uses $P\,L_A$. Self-closing $U_A(U_A)$ instead demands a tail in $U_A(U_A)$; its uninhabitedness is a separate conjecture.

13. Selected theorem statements: unproved

All 47 statements in `OpenSignaturesTheorems.v` are `Conjecture`s. The following are representative; the example typing and derived-rule claims above are also unproved.

Context validity	$\Gamma \vdash t : A \implies \Gamma\ \mathsf{wf}.$
Type correctness	$\Gamma \vdash t : A \implies \exists k.\ \Gamma \vdash A : \mathsf{Set}_k.$
Weakening	$\Gamma \vdash t : A,\ \Gamma \vdash B : \mathsf{Set}_k \implies \Gamma, x : B \vdash t : A.$
Substitution	$\Gamma, x : A \vdash t : B,\ \Gamma \vdash u : A \implies \Gamma \vdash t[u/x] : B[u/x].$
Preservation	$\Gamma \vdash t : A,\ t \to u \implies \Gamma \vdash u : A$ (also for $\rightsquigarrow$).
Progress	$\varnothing \vdash t : A \implies \mathsf{value}(t) \lor \exists u.\ t \to u.$
Normalization	$\Gamma \vdash t : A \implies t$ is strongly normalizing for $\rightsquigarrow$.
Confluence	$\Gamma \vdash t : A,\ t \rightsquigarrow^* u,\ t \rightsquigarrow^* v \implies \exists w.\ u \rightsquigarrow^* w \land v \rightsquigarrow^* w.$
Consistency	$\neg\exists t.\ \varnothing \vdash t : \bot.$
Dead soundness	$\Gamma \vdash D\ \mathsf{dead}^I_X \rightsquigarrow d \implies \Gamma \vdash d : [\![D]\!]\, X \to \bot.$
Description soundness	$\Gamma \vdash D \preceq^I_X D' \rightsquigarrow q \implies \Gamma \vdash q : [\![D]\!]\, X \to [\![D']\!]\, X.$
Coercion soundness	$\Gamma \vdash A \sqsubseteq B \rightsquigarrow c \implies \Gamma \vdash c : A \to B.$
Synthesis soundness	$\Gamma \vdash e \Rightarrow A \rightsquigarrow t \implies \Gamma \vdash t : A.$
Checking soundness	$\Gamma \vdash e \Leftarrow A \rightsquigarrow t \implies \Gamma \vdash t : A.$
Coercion coherence	$\Gamma \vdash A \sqsubseteq B \rightsquigarrow c,\ \Gamma \vdash A \sqsubseteq B \rightsquigarrow d \implies c \approx_{A \to B} d.$
Checking coherence	$\Gamma \vdash e \Leftarrow A \rightsquigarrow t,\ \Gamma \vdash e \Leftarrow A \rightsquigarrow u \implies t \approx_A u.$
Roll/unroll	$\Gamma \vdash x : C_{F,G}(i) \implies \mathsf{in}(\mathsf{unroll}_{F,G,i}\, x) \approx_{C_{F,G}(i)} x.$

The last statement additionally requires $\Gamma \vdash I : \mathsf{Set}_0$, $F, G : \mathsf{Def}\, I$, and $i : I$. Weakening uses a fresh x. $\approx_A$ quantifies over typed closing substitutions and closed Boolean observations. It is not definitional equality.

$$\Gamma \vdash A : \mathsf{Set}_0 \implies \Gamma \vdash \mathsf{NonEmpty}\, A \sqsubseteq \mathsf{List}\, A \rightsquigarrow \mathsf{toList},$$
$$\neg\exists c.\ \varnothing \vdash c : \Pi A : \mathsf{Set}_0.\, \mathsf{List}\, A \to \mathsf{NonEmpty}\, A,$$
$$\varnothing \vdash A : \mathsf{Set}_0 \implies \neg\exists x.\ \varnothing \vdash x : C_{U_A, U_A}(\star).$$

Ten closed computation checks in `OpenSignaturesExamples.v` are proved by reflexivity. They do not prove the general typing claims.

Source map.

Sections	Rocq source
1–5	`OpenSignaturesCore.v`
6–10	`OpenSignaturesElaboration.v`
11–12	`OpenSignaturesExamples.v`
13	`OpenSignaturesTheorems.v`

14. Sources and attribution

Source	Use in this document
Dagand–McBride [1]	Figures 1–4: the base dependent calculus, enumerations, descriptions, their interpretation, and the reference indexed fixed point. The version cited is arXiv v2, matching the local PDF used in the notes.
Chapman et al. [2]	Supporting source reached through EID's account of generic induction. The explicit `iAll/hyps` equations here are the Rocq reconstruction; the bottom case belongs to this revision.
Campos [3]	The discussion's constructor-subtyping manuscript, especially Section 2, Rules 1–7, and optional $\beta\eta\varphi$ conversion: motivation for restricting the outer constructor and omitting impossible clauses. Its phantom signatures and metatheorems are not imported here.

The direct $'\bot$ code, reusable open signatures, $\mathsf{close}\,F\,G$, and generated coercions are the design of this discussion. The references do not establish their metatheory.

References

[1] Pierre-Évariste Dagand and Conor McBride. *Elaborating Inductive Definitions*. 2012. arXiv:1210.6390v2, 30 October 2012. arXiv record; version cited. Local copy: `~/Workspace/Subtyping-Theory/first-class-inductive-types.pdf`.

[2] James Chapman, Pierre-Évariste Dagand, Conor McBride, and Peter Morris. *The Gentle Art of Levitation*. Proceedings of ICFP 2010, pp. 3–14. doi:10.1145/1863543.1863547; author's PDF. Supporting reference cited by EID for generic induction.

[3] Tiago Campos. *First-Class Constructor Subsets for Pattern Matching on Indexed Families*. Undated working manuscript; local source consulted 9 September 2026. Local manuscript source: `~/Workspace/Subtyping-Theory/subindex/LIPIcs_2021_Version_Sample/subtyping_paper.tex`. The template directory name is not a publication date or venue claim.